

Далее мы начнем двигаться к более значимому образу. [yt-dlp](#) — это программа, которая загружает видео с YouTube и [Imgur](#). Давайте добавим его в образ — но на этот раз мы изменим наш процесс. Вместо нашего текущего процесса, где мы добавляем вещи в Dockerfile и надеемся, что это сработает, давайте попробуем другой подход. На этот раз мы откроем интерактивную сессию и протестируем вещи, прежде чем "сохранить" их в нашем Dockerfile.

Следуя [инструкциям по установке yt-dlp](#), мы начнем следующим образом:

```
$ docker run -it ubuntu:22.04

root@8c587232a608:/# curl -L https://github.com/yt-dlp/yt-dlp/releases/latest/download/yt-dlp -o /usr/local/bin/yt-dlp
bash: curl: command not found
```

...и, как мы уже знаем, curl не установлен — давайте добавим curl с помощью apt-get снова.

```
$ apt-get update && apt-get install -y curl
$ curl -L https://github.com/yt-dlp/yt-dlp/releases/latest/download/yt-dlp -o /usr/local/bin/yt-dlp
```

В какой-то момент вы могли заметить, что sudo также не установлен, но поскольку мы root, нам это не нужно.

Далее мы добавим разрешения и запустим загруженный бинарный файл:

```
$ chmod a+rx /usr/local/bin/yt-dlp
$ yt-dlp
/usr/bin/env: 'python3': No such file or directory
```

Хорошо, [документация](#) упоминает, что для запуска yt-dlp требуется Python 3.8 или позже. Итак, давайте установим его:

```
$ apt-get install -y python3
```

Теперь мы можем попробовать запустить приложение снова:

```
$ yt-dlp

Usage: yt-dlp [OPTIONS] URL [URL...]

yt-dlp: error: You must provide at least one URL.
Type yt-dlp --help to see a list of all options.
```

Оно работает, нам просто нужно дать ему URL.

Итак, теперь, когда мы точно знаем, что нам нужно. Начиная с ubuntu:22.04, мы добавим вышеуказанные шаги в наш Dockerfile. Мы всегда должны стараться держать строки, наиболее подверженные изменениям, внизу, добавляя инструкции вниз, мы можем сохранить наши кэшированные слои — это удобная практика для ускорения процесса сборки, когда в Dockerfile есть

трудоемкие операции, такие как загрузки. Мы также добавили WORKDIR, который обеспечит загрузку видео туда.

```
FROM ubuntu:22.04

WORKDIR /mydir

RUN apt-get update && apt-get install -y curl python3
RUN curl -L https://github.com/yt-dlp/yt-dlp/releases/latest/download/yt-dlp -o /usr/local/bin/yt-dlp
RUN chmod a+x /usr/local/bin/yt-dlp

CMD ["/usr/local/bin/yt-dlp"]
```

Мы также переопределили `bash` как команду нашего образа (установленную на базовом образе) с `yt-dlp` самим. Это не совсем сработает, но давайте посмотрим, почему.

Давайте теперь соберем Dockerfile как образ `yt-dlp` и запустим его:

```
$ docker build -t yt-dlp .
...

$ docker run yt-dlp

Usage: yt-dlp [OPTIONS] URL [URL...]

yt-dlp: error: You must provide at least one URL.
Type yt-dlp --help to see a list of all options.
```

Пока все хорошо. Естественный способ использовать этот образ — дать URL как аргумент, но, к сожалению, это не работает:

```
$ docker run yt-dlp https://www.youtube.com/watch?v=uTZSILGTskA

docker: Error response from daemon: failed to create task for container: failed to create shim task: OCI runtime create failed: runc create failed: unable to start container process: exec: "https://www.youtube.com/watch?v=uTZSILGTskA": stat https://www.youtube.com/watch?v=uTZSILGTskA: no such file or directory: unknown.
ERRO[0000] error waiting for container: context canceled
```

Как мы теперь знаем, аргумент, который мы дали, заменяет команду или `CMD` :

```
$ docker run -it yt-dlp ps
PID TTY          TIME CMD
  1 pts/0        00:00:00 ps
$ docker run -it yt-dlp ls -l
total 0
$ docker run -it yt-dlp pwd
/mydir
```

Нам нужен способ иметь что-то *перед* командой. К счастью, у нас есть способ сделать это: мы можем использовать [ENTRYPOINT](#), чтобы определить основной исполняемый файл, а затем Docker объединит наши аргументы запуска для него.

```
FROM ubuntu:22.04

WORKDIR /mydir

RUN apt-get update && apt-get install -y curl python3
RUN curl -L https://github.com/yt-dlp/yt-dlp/releases/latest/download/yt-dlp -o /usr/local/bin/yt-dlp
RUN chmod a+x /usr/local/bin/yt-dlp

# Replacing CMD with ENTRYPOINT
ENTRYPOINT ["/usr/local/bin/yt-dlp"]
```

И теперь это работает как должно:

```
$ docker build -t yt-dlp .
$ docker run yt-dlp https://www.youtube.com/watch?v=XsqlHHTGQrw
[youtube] Extracting URL:https://www.youtube.com/watch?v=XsqlHHTGQrw
[youtube] uTZSILGTskA: Downloading webpage
[youtube] uTZSILGTskA: Downloading ios player API JSON
[youtube] uTZSILGTskA: Downloading android player API JSON
[youtube] uTZSILGTskA: Downloading m3u8 information
[info] uTZSILGTskA: Downloading 1 format(s): 22
[download] Destination: Master's Programme in Computer Science | University of Helsinki [XsqlHHTGQrw].mp4
[download] 100% of 6.29MiB in 00:00:00 at 9.95MiB/s
```

С `ENTRYPOINT` `docker run` теперь выполнил комбинированную команду `/usr/local/bin/yt-dlp https://www.youtube.com/watch?v=uTZSILGTskA` внутри контейнера!

`ENTRYPOINT` против `CMD` может сбивать с толку — в правильно настроенном образе, таком как наш `yt-dlp`, команда представляет список аргументов для `entrypoint`. По умолчанию `entrypoint` в Docker установлен как `/bin/sh -c`, и это передается, если `entrypoint` не установлен. Вот почему передача пути к файлу скрипта как `CMD` работает: вы передаете файл как параметр для `/bin/sh -c`.

Если образ определяет оба, то `CMD` используется для передачи [аргументов по умолчанию](#) для `entrypoint`. Давайте теперь добавим `CMD` в Dockerfile:

```
FROM ubuntu:22.04

WORKDIR /mydir

RUN apt-get update && apt-get install -y curl python3
RUN curl -L https://github.com/yt-dlp/yt-dlp/releases/latest/download/yt-dlp -o /usr/local/bin/yt-dlp
RUN chmod a+x /usr/local/bin/yt-dlp

ENTRYPOINT ["/usr/local/bin/yt-dlp"]
```

```
# define a default argument
CMD ["https://www.youtube.com/watch?v=Aa55RKWZxxI"]
```

Теперь (после повторной сборки) образ можно запускать без аргументов для загрузки видео, определенного в CMD:

```
$ docker run yt-dlp

[youtube] Extracting URL: https://www.youtube.com/watch?v=Aa55RKWZxxI
[youtube] Aa55RKWZxxI: Downloading webpage
[youtube] Aa55RKWZxxI: Downloading ios player API JSON
[youtube] Aa55RKWZxxI: Downloading android player API JSON
...
[download] 100% of 5.60MiB in 00:00:00 at 7.91MiB/s
```

Аргумент, определенный CMD, можно *переопределить*, дав один в командной строке:

```
$ docker run yt-dlp https://www.youtube.com/watch?v=DptFY_MszQs
[youtube] Extracting URL: https://www.youtube.com/watch?v=DptFY_MszQs
[youtube] DptFY_MszQs: Downloading webpage
[youtube] DptFY_MszQs: Downloading ios player API JSON
[youtube] DptFY_MszQs: Downloading android player API JSON
[youtube] DptFY_MszQs: Downloading player 9bb09009
[youtube] DptFY_MszQs: Downloading m3u8 information
[info] DptFY_MszQs: Downloading 1 format(s): 22
[download] Destination: Welcome to Kumpula campus! | University of Helsinki
[DptFY_MszQs].mp4
[download] 100% of 29.92MiB in 00:00:04 at 7.10MiB/s
```

В дополнение ко всему виденному, есть два способа установить ENTRYPOINT и CMD: **exec** форма и **shell** форма. Мы использовали exec форму, где сама команда выполняется. В shell форме команда, которая выполняется, обернута в `/bin/sh -c` — это полезно, когда вам нужно оценивать переменные окружения в команде, такие как `$MYSQL_PASSWORD` или подобные.

В shell форме команда предоставляется как строка без скобок. В exec форме команда и ее аргументы предоставляются как список (со скобками), см. таблицу ниже:

Dockerfile	Результирующая команда
ENTRYPOINT /bin/ping -c 3 CMD localhost	/bin/sh -c '/bin/ping -c 3' /bin/sh -c localhost
ENTRYPOINT ["/bin/ping","-c","3"] CMD localhost	/bin/ping -c 3 /bin/sh -c localhost
ENTRYPOINT /bin/ping -c 3 CMD ["localhost"]	/bin/sh -c '/bin/ping -c 3' localhost
ENTRYPOINT ["/bin/ping","-c","3"] CMD ["localhost"]	/bin/ping -c 3 localhost

Поскольку команда в конце Docker run будет CMD, мы хотим использовать ENTRYPOINT, чтобы указать, что запускать, а CMD, чтобы указать, какую команду (в нашем случае url) запускать.

Большую часть времени мы можем игнорировать ENTRYPOINT при сборке наших образов и использовать только CMD. Например, образ Ubuntu по умолчанию устанавливает ENTRYPOINT в bash, так что нам не нужно беспокоиться об этом. И это дает нам удобство позволять нам легко перезаписывать CMD, например, с bash, чтобы войти внутрь контейнера.

Мы можем протестировать, как некоторые другие проекты делают это. Давайте попробуем Python:

```
$ docker pull python:3.11
...
$ docker run -it python:3.11
Python 3.11.8 (main, Feb 13 2024, 09:03:56) [GCC 12.2.0] on linux
Type "help", "copyright", "credits" or "license" for more information.
>>> print("Hello, World!")
Hello, World!
>>> exit()

$ docker run -it python:3.11 --version
docker: Error response from daemon: OCI runtime create failed:
container_linux.go:370: starting container process caused: exec: "--version":
executable file not found in $PATH: unknown.

$ docker run -it python:3.11 bash
root@1b7b99ae2f40:/#
```

Из этого эксперимента мы узнали, что у них есть ENTRYPOINT, отличный от Python, но CMD — это Python, и мы можем перезаписать его, здесь с помощью bash. Если бы у них был ENTRYPOINT как Python, мы могли бы запустить `--version`. Мы можем создать свой собственный образ для личного использования, как мы делали в предыдущем упражнении с новым Dockerfile:

```
FROM python:3.11
ENTRYPOINT ["python3"]
CMD ["--help"]
```

Результат — образ, который имеет Python как ENTRYPOINT, и вы можете добавлять команды в конце, например, `--version`, чтобы увидеть версию. Без перезаписи команды он выведет справку.

Теперь у нас есть две проблемы с проектом yt-dlp:

- Серьезная: Загруженные файлы остаются в контейнере
- Незначительная: Наш процесс сборки контейнера создает много слоев, что приводит к увеличению размера образа

Мы исправим серьезную проблему сначала. Незначительная проблема получит наше внимание в части 3.

Проверяя `docker container ls -a`, мы можем видеть все наши предыдущие запуски. Когда мы фильтруем этот список с помощью

```
$ docker container ls -a --last 3
```

CONTAINER ID	IMAGE	COMMAND	CREATED
STATUS	PORTS	NAMES	
be9fdbcafb23	yt-dlp	"/usr/local/bin/yout..."	Less than a second ago
Exited (0)	About a minute ago	determined_elion	
b61e4029f997	f2210c2591a1	"/bin/sh -c \"/usr/lo..."	Less than a second ago
Exited (2)	About a minute ago	vigorous_bardeen	
326bb4f5af1e	f2210c2591a1	"/bin/sh -c \"/usr/lo..."	About a minute ago
Exited (2)	3 minutes ago	hardcore_carson	

Мы видим, что последний контейнер был `be9fdbcafb23` или `determined_elion` для нас, людей.

```
$ docker diff determined_elion
C /root
A /root/.cache
A /root/.cache/yt-dlp
A /root/.cache/yt-dlp/youtube-nsig
A /root/.cache/yt-dlp/youtube-nsig/9bb09009.json
C /mydir
A /mydir/Welcome to Kumpula campus! | University of Helsinki [DptFY_MszQs].mp4
```

Давайте попробуем команду `docker cp`, чтобы скопировать файл из контейнера на хост-машину. Мы должны использовать кавычки сейчас, так как имя файла содержит пробелы.

```
$ docker cp "determined_elion://mydir/Welcome to Kumpula campus! | University of Helsinki [DptFY_MszQs].mp4" .
```

И теперь у нас есть файл локально, и мы можем смотреть его, если на машине установлен подходящий плеер. К сожалению, использование `docker cp` не является правильным способом исправить нашу проблему. В следующем разделе мы улучшим это.

Улучшенный curler

С помощью `ENTRYPOINT` мы можем сделать curler из [Упражнения 1.7](#) более гибким.

Измените скрипт так, чтобы он принимал первый аргумент как ввод:

```
#!/bin/bash

echo "Searching..";
sleep 1;
curl http://$1;
```

И измените CMD на `ENTRYPOINT` с форматом `["./script.sh"]`. Теперь мы можем запустить

```
$ docker build . -t curler-v2
$ docker run curler-v2 helsinki.fi
```

Searching..

% Total	% Received	% Xferd	Average Speed	Time	Time	Time	Current
			Dload Upload	Total	Spent	Left	Speed
100	232	100	232	0	0	13647	0
--:--:--	--:--:--	--:--:--	--:--:--	--:--:--	--:--:--	--:--:--	13647

<!DOCTYPE HTML PUBLIC "-//IETF//DTD HTML 2.0//EN">

<html><head>

<title>301 Moved Permanently</title>

</head><body>

<h1>Moved Permanently</h1>

<p>The document has moved here.</p>

</body></html>